



Projeto GRAB!T

RELATÓRIO DE PROGRAMAÇÃO DE APLICAÇÃO DO
LADO DO CLIENTE

GUSTAVO SANTOS – 27964 / ANDRÉ NORONHA – 27976 / FILIPE
LINO – 27978

Conteúdo

1	Introdução	3
2	Revisão da aplicação desktop	4
3	Organização do trabalho e ferramentas	5
4	Análise do problema.....	6
4.1	Sistemas Relacionados	6
4.1.1	Aplicação 1 – Uber Eats.....	6
4.1.2	Aplicação 2 – iFood.....	7
4.1.3	Aplicação 3 – Glovo	8
4.1.4	Aplicação 4 – Bolt Food	9
4.2	Caracterização dos Utilizadores	9
4.2.1	Perfil de utilizador 1 (Cliente Final) – João Pedro	9
4.2.2	Perfil de utilizador 2 (Empresária) – Ana Martins	10
4.2.3	Perfil de utilizador 3 (Estafeta) – Miguel Santos	10
4.2.4	Perfil de utilizador 4 (Cliente Tradicional) – Rui Costa	10
4.3	Cenários de Utilização	11
4.3.1	Cenário 1 – Realizar inscrição de utilizador.....	11
4.3.2	Cenário 2 – Efectuar pedido de refeição rápida (Cliente).....	11
4.3.3	Cenário 3 - Adicionar um prato à ementa (Restaurante).....	12
4.3.4	Cenário 4 - Confirmação de Entrega e Atualização de Saldo (Estafeta)	12
5	Funcionalidades da aplicação.....	12
6	Desenho da interface da aplicação.....	13
7	Desenho da base de dados	14
8	Conceção da base de dados	16
8.1	Query de Atualização de Saldo (ClientDao)	16
8.2	Listagem de Pedidos para o Estafeta (OrderDao)	17
8.3	Aceitação de Pedido pelo Estafeta (OrderDao)	17
8.4	Cálculo Matemático de Ganhos (OrderDao)	17
8.5	Alteração de Stock (ProductDao)	18
9	Programação da lógica da aplicação	18
9.1	Navegação Centralizada (NavHost)	18

9.2	Gestão de Estado Reativa (State Management).....	19
9.3	Programação Assíncrona e Segurança (Corrotinas).....	19
10	Testes com a aplicação	20
10.1	Testes de Utilização: Vertente Cliente Final (Conduzidos por Gustavo Santos)	20
	Teste 1.1: Realização de uma Encomenda (Avaliador Externo 1)	20
	Teste 1.2: Pesquisa e Filtros de Restaurantes (Avaliador Externo 2)	20
	Teste 1.3: Edição do Carrinho de Compras (Avaliador Interno 1)	21
	Teste 1.4: Consulta de Histórico e Repetição de Pedido (Avaliador Interno 2).....	21
10.2	Testes de Utilização: Vertente Empresarial/Restaurante (Conduzidos por André Noronha)	22
	Teste 2.1: Adição de Novo Prato à Ementa (Avaliador Externo 1).....	22
	Teste 2.2: Atualização de Disponibilidade de um Produto (Avaliador Externo 2)	22
	Teste 2.3: Receção e Aceitação de um Novo Pedido (Avaliador Interno 1)	22
	Teste 2.4: Alteração do Estado do Pedido para "Pronto" (Avaliador Interno 2).....	23
10.3	Testes de Utilização: Vertente Estafeta (Conduzidos por Filipe Lino).....	23
	Teste 3.1: Ativação de Perfil e Aceitação de Entrega (Avaliador Externo1).....	23
	Teste 3.2: Consulta de Rota e Confirmação de Recolha (Avaliador Externo 2)	23
	Teste 3.3: Finalização da Entrega e Verificação de Ganhos (Avaliador Interno 1)	24
	Teste 3.4: Alteração de Estado para Indisponível (Avaliador Interno 2)	24
10.4	Teste 4: Vertente Estafeta (Levantamento de Fundos)	25
11	Página web da aplicação e screencast	26
12	Conclusão.....	27

1 Introdução

No contexto atual, o acesso rápido e móvel à informação é um requisito fundamental. O projeto GRAB!T nasce com o objetivo central de oferecer uma solução digital interativa, rápida e intuitiva no setor das entregas ao domicílio (delivery), focando-se num maior nível de segurança e fluidez no ato da entrega. Através da transição de uma solução previamente pensada para desktop para o ambiente móvel nativo, pretendemos oferecer aos utilizadores uma ferramenta perfeitamente adaptada às suas rotinas dinâmicas.

Os objetivos principais desta aplicação passam por desenvolver uma solução estruturada, utilizando a ferramenta Android Studio, a linguagem Kotlin e a moderna framework Jetpack Compose. A aplicação liga de forma eficiente três vertentes distintas: o Cliente Final, o Restaurante (parceiro) e o Estafeta, permitindo gerir ementas, submeter pedidos e acompanhar entregas, suportada por uma base de dados robusta em SQL.

O presente relatório encontra-se organizado em doze capítulos. No capítulo 2, é descrita a revisão à aplicação desktop. O capítulo 3 detalha a organização do trabalho de equipa. O capítulo 4 apresenta a análise do problema, explorando as aplicações concorrentes (ex: Uber Eats, iFood, Glovo), perfis de utilizadores e cenários. No capítulo 5 listam-se as funcionalidades. O desenho da interface gráfica é abordado no capítulo 6, seguindo-se o desenho (cap. 7) e conceção (cap. 8) da base de dados. O capítulo 9 explica a programação da lógica em Kotlin. O capítulo 10 descreve os rigorosos testes de usabilidade realizados. Por fim, no capítulo 11 é apresentada a página web promocional e o screencast, terminando com a conclusão no capítulo 12.

2 Revisão da aplicação desktop

No semestre anterior, o desenvolvimento da aplicação desktop permitiu à equipa estabelecer as regras de negócio base e definir os três perfis vitais do ecossistema GRAB!T. O trabalho em equipa foi estruturado dividindo o planeamento técnico, estruturação de dados e idealização gráfica.

As funcionalidades desenvolvidas revelaram-se cruciais para validar a lógica relacional. No entanto, sentimos dificuldades inerentes à própria plataforma: desenvolver um sistema de entregas em ambiente desktop não reflete a realidade do mercado, uma vez que clientes e, sobretudo, estafetas necessitam de mobilidade constante. O ecossistema de delivery exige respostas imediatas no terreno.

Avaliando esse percurso, a nossa abordagem para a aplicação móvel consistiu numa reconstrução total orientada ao Design Centrado no Utilizador (UCD). O foco passou a ser a redução drástica de interações (cliques) necessárias para efetuar ou aceitar um pedido, abandonando o XML clássico e tirando partido do Jetpack Compose para apresentar interfaces modernas e atualizações de estado em tempo real nos ecrãs dos smartphones.

3 Organização do trabalho e ferramentas

Foi criado o repositório no GitHub com o seguinte endereço:

<https://github.com/lionhrino/Grabit>. Neste repositório pode ser encontrado o relatório do projeto, a apresentação final, o screencast, o desenho da base de dados, o website e o código da aplicação móvel.

No decorrer do projeto foram utilizadas as seguintes ferramentas:

- **Android Studio (Kotlin e Jetpack Compose):** Para o desenvolvimento nativo, criação visual da interface reativa (UI) e gestão de estado (State Management).
- **Room (SQL):** Para a estruturação, interligação e persistência da base de dados relacional.
- **Discord e GitHub:** Para reuniões diárias, comunicação e controlo de versões de código.
- **HTML/CSS:** Para o desenvolvimento do website promocional da aplicação.

O trabalho presencial e remoto foi sustentado por uma distribuição clara de tarefas: o Gustavo Santos focou-se na estruturação lógica Front-End e Backend, navegação nativa e Jetpack Compose (vertente Cliente); o André Noronha assumiu a Identidade Visual (Design de logótipos e protótipos) e a vertente Empresarial/Restaurantes; e o Filipe Lino encarregou-se da arquitetura de Backend/Base de Dados (SQL), Vertente do Estafeta e organização da documentação.

4 Análise do problema

Neste capítulo, apresentamos os sistemas relacionados que estudámos, assim como os diferentes perfis de utilizador identificados. Por fim, destacamos os cenários de utilização da aplicação que considerámos mais relevantes.

4.1 Sistemas Relacionados

Foram analisadas quatro grandes plataformas no mercado atual de *delivery* para identificar padrões de usabilidade e mecânicas de base de dados. De seguida, apresentamos os aspetos positivos e negativos identificados.

4.1.1 Aplicação 1 – Uber Eats

- Apresentação: Plataforma multinacional líder na entrega de refeições e produtos ao domicílio, focada em ligar clientes, restaurantes e estafetas.
- Aspetos positivos e negativos: Como aspetos positivos destacam-se a facilidade de utilização, enorme variedade de oferta e o processo de pagamento simplificado. Nos aspetos negativos, apontamos a existência de custos ou taxas adicionais muitas vezes não previstos pelo utilizador, e a disponibilidade geográfica limitada.

Figura 1: Site Uber Eats



Figura 1 – Uber Eats (site)

4.1.2 Aplicação 2 – iFood

- Apresentação: Uma das maiores plataformas de entrega e descoberta de comida da América do Sul.
- Aspectos positivos e negativos: Destaca-se positivamente o excelente acompanhamento de encomendas em tempo real e o robusto sistema de avaliações. Como pontos negativos, identificámos as variações dos custos de entrega e um forte excesso de conteúdo promocional poluidor da interface.

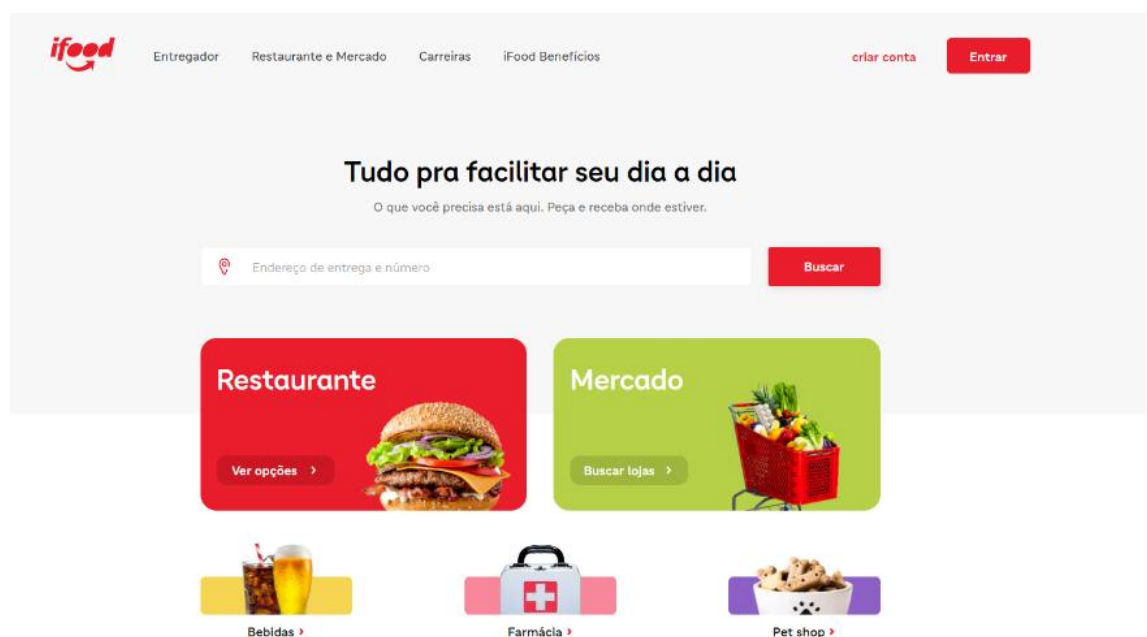


Figura 2 – ifood(site)

4.1.3 Aplicação 3 – Glovo

- Apresentação: Aplicação de entregas a pedido (on-demand) estruturada através de uma navegação por categorias circulares visuais.
- Aspectos positivos e negativos: Apresenta um design limpo e uma curva de aprendizagem extremamente reduzida (positivo). No entanto, peca pela falta de consistência na formatação visual das ementas entre diferentes restaurantes e tem forte dependência de funcionalidades bloqueadas por taxas (negativo).

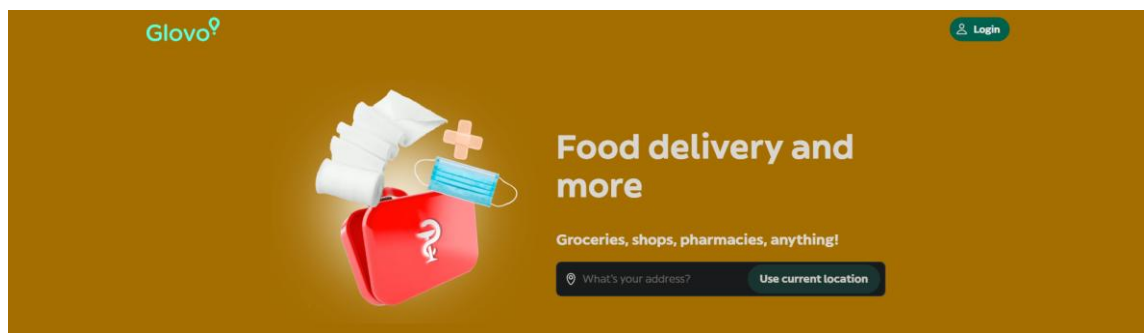


Figura 3 – Glovo (site)

4.1.4 Aplicação 4 – Bolt Food

- Apresentação: Plataforma europeia de entrega de comida que tira partido da sua rede de mobilidade e motoristas.
- Aspetos positivos e negativos: Oferece excelente acompanhamento de rotas e agressividade nas promoções (positivo). Contudo, o sistema de filtragem de pratos e personalização de ingredientes é frequentemente mais restrito do que na concorrência direta (negativo).



Figura 4 – Bolt Food (site)

4.2 Caracterização dos Utilizadores

Para modelar o funcionamento da GRAB!T, identificámos quatro personas que representam as vertentes do nosso sistema (no âmbito das cadeiras de Programação e Sistemas Interativos).

4.2.1 Perfil de utilizador 1 (Cliente Final) – João Pedro



Figura 5 - Fotografia usada como referência à persona

O João tem 21 anos e é estudante universitário. Passa os dias focado em projetos académicos de programação, com pouco tempo para planear refeições. Procura conveniência e eficiência: precisa de uma app reativa para encomendar comida de forma imediata sem banners intrusivos. Não tem tolerância para checkouts demorados ou taxas que não sejam explícitas logo no ecrã inicial.

4.2.2 Perfil de utilizador 2 (Empresária) – Ana Martins



Figura 5 - Fotografia usada como referência à persona

A Ana tem 38 anos e gere um restaurante de comida tradicional. Aderiu ao delivery para aumentar as vendas, mas não tem tempo a perder com tecnologia complexa. Precisa de um painel de gestão empresarial simples para conseguir atualizar a ementa, alterar preços, ativar/desativar pratos e processar as encomendas sem causar o caos na cozinha.

4.2.3 Perfil de utilizador 3 (Estafeta) – Miguel Santos



Figura 5 - Fotografia usada como referência à persona

O Miguel tem 24 anos e trabalha como estafeta em part-time para conciliar com os estudos. Valoriza a flexibilidade de horários, mas a sua principal dor é a dependência de interfaces falíveis. Precisa de ecrãs diretos (com botões grandes) que lhe permitam aceitar pedidos em trânsito e visualizar a sua rota e ganhos sem erros de comunicação com a base de dados.

4.2.4 Perfil de utilizador 4 (Cliente Tradicional) – Rui Costa



Figura 6 - Fotografia usada como referência à persona

O Rui tem 52 anos, é encarregado de obra e passa os dias no estaleiro. Ouve falar da GRAB!T através de colegas mais novos e decide instalar a aplicação para pedir almoços rápidos para a equipa. Tem muito pouca paciência para burocracias tecnológicas e desiste facilmente se as aplicações exigirem muitos passos. Precisa de um *design* extremamente acessível: fontes legíveis, botões grandes com cores óbvias (como o verde para avançar) e formulários curtos. Acima de tudo, valoriza a prevenção de erros, necessitando de avisos claros caso se engane a digitar a morada ou a palavra-passe, para não se sentir frustrado.

4.3 Cenários de Utilização

Foram desenvolvidos quatro cenários cruciais para testar o fluxo interativo e lógico da aplicação.

4.3.1 Cenário 1 – Realizar inscrição de utilizador

O Rui instala a aplicação e seleciona a opção "Criar Conta". Sendo um utilizador menos experiente, beneficia da interface limpa e escolhe o seu perfil. Começa a preencher os dados vitais. A aplicação ajuda-o realizando validações nativas em tempo real (ex: avisando imediatamente se o email for inválido). Após submeter o formulário com sucesso, o backend (SQL) guarda as informações de forma segura. O Rui efetua o Login e a aplicação transita de forma suave para o ecrã principal, pronto a encomendar o seu primeiro almoço.

Assim a vossa história fica muito mais coesa: têm 4 utilizadores perfeitamente enquadrados no mundo real do delivery, e todos eles executam ações fundamentais na vossa base de dados!

4.3.2 Cenário 2 – Efectuar pedido de refeição rápida (Cliente)

O João Pedro abre a GRAB!T, seleciona a categoria de "Hambúrgueres" no ecrã e escolhe o seu restaurante favorito. Adiciona dois pratos ao carrinho (com a UI a atualizar o preço total dinamicamente). No ecrã de *checkout*, verifica o resumo detalhado das taxas de entrega e clica em "Confirmar Pagamento", sendo reencaminhado para o ecrã de rastreamento do estafeta.

4.3.3 Cenário 3 - Adicionar um prato à ementa (Restaurante)

A Ana Martins inicia sessão no seu perfil empresarial. Navega até à área da "Gestão de Cardápio" e clica no botão "+". Insere o nome do prato e o preço, verificando as informações. Ao clicar em "Guardar", o sistema invoca a lógica de Base de Dados, inserindo a refeição na tabela e tornando-a visível de imediato para a pesquisa dos clientes.

4.3.4 Cenário 4 - Confirmação de Entrega e Atualização de Saldo (Estafeta)

O Miguel faz login na parte dos clientes. Recebe uma notificação na "Central de Estafetas" e seleciona "Aceitar Entrega". Dirige-se ao restaurante, levanta a refeição e, após concluir a rota, confirma a entrega na app. O sistema altera o estado da encomenda no SQL e credita automaticamente o respetivo valor financeiro na sua "Carteira GRAB!T".

5 Funcionalidades da aplicação

A versão móvel da GRAB!T integra nativamente as seguintes funcionalidades:

1. Autenticação segmentada (Registo e Login dinâmico com validação e ocultação de credenciais).
2. Dashboard reativo com navegação estruturada via Bottom Navigation Bar.
3. Gestão e State Management do Carrinho de Compras (feedback imediato ao adicionar itens).
4. Catálogo SQL em tempo real: pesquisa, listagem e gestão de pratos pelas Empresas.
5. Painel de Receção de Pedidos e alteração de estados da cozinha.
6. Central do Estafeta para alocação de serviços de entrega.
7. Carteira Digital (Wallet) e Histórico de Utilizador para visualização de ganhos e encomendas passadas.

6 Desenho da interface da aplicação

O desenho da interface pautou-se pelos rigorosos princípios de Sistemas Interativos: as **8 Regras de Shneiderman** (manter a consistência gráfica e fornecer feedback informativo via Snackbars), as **10 Heurísticas de Nielsen** (prevenção de erros mantendo botões inativos enquanto campos faltarem) e os **Princípios de Norman**. A identidade visual, incluindo logótipos simplificados e detalhados, assumiu cores neutras e botões de destaque verdes.

A grande inovação arquitetónica residiu na implementação final no Android Studio através do **Jetpack Compose**. A adoção desta framework declarativa moderna dispensou a gestão clássica em layouts XML, permitindo construir componentes visuais altamente reativos e modulares, garantindo transições suaves e hierarquias claras.

(DICA: Inserir aqui 2 ou 3 Print Screens da vossa app, como o ecrã Home, Carrinho ou Central de Estafetas, presentes no PDF).

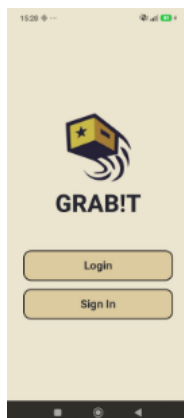


Figura 7 – Tela de login

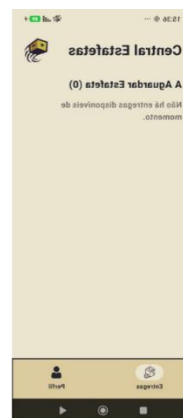


Figura 8 – Central de Estafetas

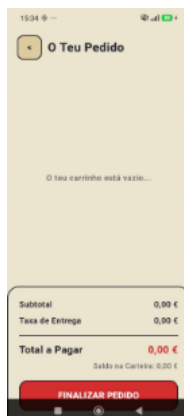


Figura 9 – Carrinho de compras

7 Desenho da base de dados

Para garantir o correto funcionamento da aplicação no ambiente mobile, o modelo conceptual foi estruturado com vista a uma relação eficiente entre os três perfis de utilizador (Cliente, Restaurante e Estafeta) e as lógicas de comércio (e-commerce).

As principais entidades identificadas e mapeadas para tabelas foram:

- **table_clients:** Gestão de dados, autenticação e saldo dos clientes.
- **table_restaurants:** Informações comerciais e de login dos parceiros.
- **table_deliveries:** Registo de atividade e autenticação dos estafetas.
- **table_products:** O catálogo de pratos oferecidos, relacionado com o ID do restaurante.
- **table_orders:** Registo central das encomendas submetidas.
- **table_order_items:** Os detalhes específicos de cada item dentro de um pedido.

As relações foram estruturadas para evitar redundância e assegurar respostas rápidas da aplicação, permitindo transições de estado fluídas (ex: de "A Aguardar Estafeta" para "Em Caminho") sem sobrecarregar a memória do dispositivo.



Figura 10 – Diagrama da Base de Dados

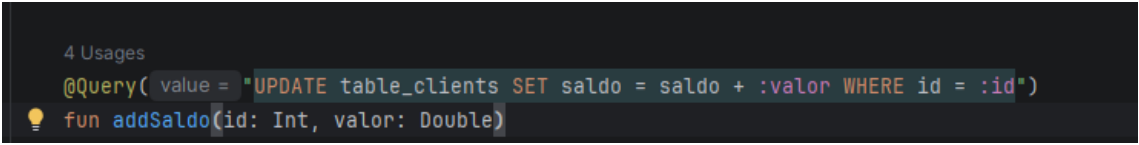
8 Conceção da base de dados

A conceção física da base de dados foi implementada recorrendo à biblioteca Room, que funciona como uma robusta camada de abstração sobre o SQLite nativo do Android. Esta abordagem garantiu uma tipagem forte e a gestão controlada de versões do esquema da base de dados (atualmente na versão 4, atualizada especificamente para integrar o sistema de pedidos e a gestão de stock).

As tabelas foram geradas através da anotação de Entidades nas classes modelo (Client, Delivery, Restaurant, Product, Order e OrderItem) e as operações de manipulação de dados (CRUD) foram estruturadas em Data Access Objects (DAOs). Para garantir interfaces sempre atualizadas, a arquitetura fez uso de Kotlin Flows (Flow<T>), permitindo que a aplicação reaja de imediato a qualquer alteração nos dados (ex: novos pedidos).

Abaixo apresentamos exemplos das queries e comandos reais desenvolvidos em Kotlin, com uma breve explicação do seu papel no sistema:

8.1 Query de Atualização de Saldo (ClientDao)



```
4 Usages
@Query(value = "UPDATE table_clients SET saldo = saldo + :valor WHERE id = :id")
fun addSaldo(id: Int, valor: Double)
```

Figura 11 – Add Saldo

(Explicação: Atualiza de forma dinâmica o valor da carteira digital de um cliente na base de dados, somando o valor inserido ao seu saldo atual, identificando o utilizador através do seu ID).

8.2 Listagem de Pedidos para o Estafeta (OrderDao)

```
1 Usage
@Query( value = "SELECT * FROM table_orders WHERE estado = 'A Aguardar Estafeta'")
fun getAvailableOrdersForDelivery(): Flow<List<Order>>
```

Figura 12 – AvailableOrdersForDelivery

(Explicação: Pesquisa todos os pedidos cujo estado exija a alocação de um serviço de entrega. O uso de Flow garante que, assim que um novo pedido entra no sistema com este estado, o ecrã do estafeta é atualizado em tempo real).

8.3 Aceitação de Pedido pelo Estafeta (OrderDao)

```
1 Usage
@Query( value = "UPDATE table_orders SET estafetaId = :estafetaId, estado = 'Em Caminho' WHERE id = :pedidoId")
fun acceptDelivery(pedidoId: Int, estafetaId: Int)
```

Figura 13 – acceptDelivery

(Explicação: Numa única operação transacional, atualiza o pedido associando-lhe o ID do estafeta que aceitou o serviço, e altera imediatamente o seu estado operacional para 'Em Caminho', notificando o restaurante e o cliente).

8.4 Cálculo Matemático de Ganhos (OrderDao)

```
1 Usage
@Query( value = "SELECT COUNT(*) * 1.90 FROM table_orders WHERE estafetaId = :estafetaId AND estado = 'Entregue'")
fun getDeliveryEarnings(estafetaId: Int): Flow<Double?>
```

Figura 14 – getDeliveryEarnings

(Explicação: Demonstra a capacidade de processamento analítico da base de dados. Em vez de calcular no código lógico, a query multiplica diretamente o número total de encomendas entregues por um estafeta por uma taxa fixa de lucro de 1,90€, devolvendo o total diário).

8.5 Alteração de Stock (ProductDao)

```
@Query(value = "UPDATE table_products SET stock = :novoStock WHERE id = :produtoId")  
fun updateStock(produtoId: Int, novoStock: Int)
```

Figura 8 – updateStock

(Explicação: Permite aos restaurantes atualizar a quantidade disponível de um determinado prato, garantindo que os clientes não consigam encomendar produtos esgotados).

9 Programação da lógica da aplicação

O desenvolvimento lógico no Android Studio destacou-se pela utilização exclusiva da linguagem nativa Kotlin. O coração tecnológico da GRAB!T residiu na aplicação de State Management (Gestão de Estado) através do Jetpack Compose, na utilização de Corrotinas para assincronismo e numa navegação centralizada.

Abaixo destacamos os três blocos de código mais relevantes que demonstram a arquitetura do nosso sistema:

9.1 Navegação Centralizada (NavHost)

A transição entre os mais de 15 ecrãs da aplicação foi garantida sem recurso ao método clássico (e pesado) de abrir múltiplas Activities. Utilizámos o NavHost na MainActivity, o que permite injetar os DAOs e os IDs de sessão diretamente nas rotas, mantendo a aplicação leve (Single-Activity Architecture).

```
val navController = rememberNavController()  
// ...  
NavHost(navController = navController, startDestination =  
"initial_screen") {  
    composable("initial_screen") { InitialScreen(navController) }  
  
    // Injeção de dependências e passagem do ID da sessão atual  
    composable("client_home") {  
        ClientHomeScreen(  
            navController = navController,  
            orderDao = orderDao,  
            clientId = UserSession.currentClientId  
        )  
    }  
}
```

9.2 Gestão de Estado Reativa (State Management)

Para garantir que a interface reage instantaneamente sem necessidade de recarregar a página (por exemplo, ao adicionar um prato ao pedido), utilizámos StateFlow no CartViewModel. A interface "observa" este estado de forma contínua, recalculando valores de forma imediata e automática.

Código Implementado (CartViewModel e Ecrãs):

```
// 1. O Estado é definido no ViewModel protegendo os dados
private val _cartItems =
MutableStateFlow<List<CartItem>>(emptyList())
val cartItems: StateFlow<List<CartItem>> = _cartItems.asStateFlow()

// 2. A Interface observa o Estado e atualiza-se sozinha
val cartItems by cartViewModel.cartItems.collectAsState()
val subtotal = cartItems.sumOf { it.precoUnitario * it.quantidade }
```

9.3 Programação Assíncrona e Segurança (Corrotinas)

Para prevenir o congelamento da interface (ANR - Application Not Responding) durante acessos à base de dados SQL (Room), envolvemos a lógica em Coroutines. Operações de leitura/escrita ocorrem na thread de background (Dispatchers.IO), e a atualização visual é devolvida à thread principal (Dispatchers.Main).

Código Implementado (Lógica de Login e Registo):

```
Button(  
    onClick = {  
        if (email.isNotBlank() && password.isNotBlank()) {  
            // Lança a operação assíncrona sem bloquear a Interface  
Gráfica  
            coroutineScope.launch(Dispatchers.IO) {  
  
                // 1. Acede à Base de Dados Room (Background Thread)  
                val isClient = clientDao?.login(email, password)  
  
                if (isClient != null) {  
                    UserSession.currentClientId = isClient.id  
  
                    // 2. Volta à Main Thread para navegar para o  
próximo ecrã  
                    withContext(Dispatchers.Main) {  
                        navController.navigate("client_home")  
                    }  
                    return@launch  
                }  
            }  
        }  
    }  
)
```

10 Testes com a aplicação

Para aferir a usabilidade, o desempenho lógico e a fluidez do Jetpack Compose, realizámos testes de usabilidade com a aplicação móvel. Os testes foram divididos pelos membros do grupo, focando-se em cenários específicos para testar as três vertentes do ecossistema GRAB!T. Durante as sessões, foi utilizada a técnica Think Aloud, pedindo aos utilizadores que verbalizassem os seus pensamentos enquanto realizavam as tarefas. Resultados do desempenho estatístico:

10.1 Testes de Utilização: Vertente Cliente Final (Conduzidos por Gustavo Santos)

Teste 1.1: Realização de uma Encomenda (Avaliador Externo 1)

- Utilizador: Rute (Mãe - Pessoa de fora).
- Tarefa: Explorar a lista de restaurantes, seleccionar um prato para o carrinho e finalizar o pedido.
- Desempenho: 2 minutos e 10 segundos | 16 cliques | 0 erros.
- Observações: A utilizadora conseguiu concluir a tarefa sem hesitações. Referiu que as imagens a ajudaram a decidir e elogiou a visibilidade do botão de "Finalizar Pedido". A inserção na base de dados ocorreu com sucesso.

Teste 1.2: Pesquisa e Filtros de Restaurantes (Avaliador Externo 2)

- Utilizador: Lourenço (Irmão - Pessoa de fora).
- Tarefa: Utilizar a barra de pesquisa para encontrar um restaurante de "Pizzas" e abrir a respetiva ementa.
- Desempenho: 45 segundos | 10 cliques | 0 erros.
- Observações: Destacou que o facto de a aplicação filtrar os resultados instantaneamente à medida que escrevia (graças à reatividade do Jetpack Compose) tornou a experiência muito rápida.

Teste 1.3: Edição do Carrinho de Compras (Avaliador Interno 1)

- Utilizador: Pedro Gonçalves (Colega da Universidade - Pessoa de dentro).
- Tarefa: Adicionar itens, alterar a quantidade e remover um prato antes do pagamento.
- Desempenho: 1 minuto e 15 segundos | 12 cliques | 1 erro (tentou fazer swipe para apagar o item).
- Observações: O erro detetado resultou numa sugestão técnica de implementar swipe-to-dismiss no futuro. Referiu que a atualização do preço total de forma imediata reflete um excelente trabalho de State Management.

Teste 1.4: Consulta de Histórico e Repetição de Pedido (Avaliador Interno 2)

- Utilizador: Tomás Santos (Colega da Universidade - Pessoa de dentro).
- Tarefa: Aceder à área de perfil através da barra de navegação e adicionar saldo à carteira digital.
- Desempenho: 35 segundos | 4 cliques | 0 erros.
- Observações: Considerou a transição pela Bottom Navigation Bar fluida e classificou o ecrã de carregamento de conta como seguro, eficiente e focado na rapidez da transação.

10.2 Testes de Utilização: Vertente Empresarial/Restaurante (Conduzidos por André Noronha)

Foram realizados testes para garantir que a gestão logística do restaurante (adição de ementas e controlo de pedidos) é à prova de falhas.

Teste 2.1: Adição de Novo Prato à Ementa (Avaliador Externo 1)

- Utilizador: Mariana (Mãe - Pessoa de fora).
- Tarefa: Adicionar um novo prato na área da ementa, inserindo nome e preço.
- Desempenho: 2 minutos e 30 segundos | 9 cliques | 1 erro (tentou submeter o prato sem preencher o preço).
- Observações: A aplicação bloqueou corretamente a submissão, demonstrando as validações nativas e a prevenção de erros. A utilizadora sugeriu que o aviso "Campo Obrigatório" tivesse um maior destaque a vermelho.

Teste 2.2: Atualização de Disponibilidade de um Produto (Avaliador Externo 2)

- Utilizador: Jorge (Irmão - Pessoa de fora).
- Tarefa: Localizar um prato existente e alterar o seu estado para "Indisponível".
- Desempenho: 1 minuto e 15 segundos | 5 cliques | 0 erros.
- Observações: O utilizador elogiou o layout intuitivo e destacou que a utilização de um botão toggle (interruptor) evita confusões no ambiente acelerado de uma cozinha.

Teste 2.3: Receção e Aceitação de um Novo Pedido (Avaliador Interno 1)

- Utilizador: Mariana Silva (Colega da Universidade - Pessoa de dentro).
- Tarefa: Identificar notificação de nova encomenda e carregar no botão para aceitar.
- Desempenho: 25 segundos | 2 cliques | 0 erros.
- Observações: A utilizadora confirmou que a estruturação e os botões amplos garantem que os funcionários do restaurante possam gerir os pedidos rapidamente em ecrãs táteis.

Teste 2.4: Alteração do Estado do Pedido para "Pronto" (Avaliador Interno 2)

- Utilizador: Tiago Mendes (Colega da Universidade - Pessoa de dentro).
- Tarefa: Alterar um pedido de "Em Preparação" para "Pronto para Recolha", alertando o estafeta.
- Desempenho: 30 segundos | 3 cliques | 0 erros.
- Observações: Referiu que a consistência visual e a transição instintiva de estados (que interação de imediato com a base de dados) ajudam o restaurante a manter-se organizado.

10.3 Testes de Utilização: Vertente Estafeta (Conduzidos por Filipe Lino)

Para avaliar a correta atualização da tabela SQL de encomendas e saldos na vertente mobile, focámo-nos na agilidade nas ruas.

Teste 3.1: Ativação de Perfil e Aceitação de Entrega (Avaliador Externo1)

- Utilizador: João (Irmão - Pessoa de fora).
- Tarefa: Abrir a app, aguardar a notificação e carregar em "Aceitar Entrega".
- Desempenho: 35 segundos | 5 cliques | 0 erros.
- Observações: Destacou que o tamanho do botão "Aceitar Entrega" facilita a utilização em movimento. A base de dados Room registou o novo estado sem atrasos.

Teste 3.2: Consulta de Rota e Confirmação de Recolha (Avaliador Externo 2)

- Utilizador: Maria (Mãe - Pessoa de fora).
- Tarefa: Consultar detalhes de recolha e confirmar levantamento da refeição no restaurante.
- Desempenho: 50 segundos | 4 cliques | 0 erros.
- Observações: A utilizadora achou muito simples identificar a morada de recolha e percebeu claramente o passo seguinte, validando que a interface é acessível.

Teste 3.3: Finalização da Entrega e Verificação de Ganhos (Avaliador Interno 1)

- Utilizador: Gonçalo Correia (Colega da Universidade - Pessoa de dentro).
- Tarefa: Marcar a entrega como concluída e aceder ao perfil para verificar a atualização do pagamento taxado.
- Desempenho: 45 segundos | 4 cliques | 0 erros.
- Observações: Confirmou que, assim que a entrega foi dada como concluída, o saldo atualizou instantaneamente no ecrã. Isto permitiu provar a integridade da tabela SQL de clientes/saldos.

Teste 3.4: Alteração de Estado para Indisponível (Avaliador Interno 2)

- Utilizador: Lucas Oliveira (Colega da Universidade - Pessoa de dentro).
- Tarefa: No final do turno, levantar o saldo feito durante o expediente.
- Desempenho: 20 segundos | 3 cliques | 0 erros.
- Observações: Considerou o processo instintivo e claro. O teste provou que a plataforma processa a atualização financeira de forma segura e rápida.

10.4 Teste 4: Vertente Estafeta (Levantamento de Fundos)

Cumprindo as diretrizes da cadeira e a metodologia de avaliação interpares adotada pela turma, realizámos testes cruzados avaliando o projeto de videovigilância dos nossos colegas (LynxSegur - Carlos Fuzil, Pedro Gonçalves e Tomás Santos), enquanto estes avaliaram a nossa aplicação GRAB!T.

Testes efetuados pela equipa GRAB!T à aplicação LynxSegur:

- **Gustavo Santos:** Tarefa: Testar o ecrã de alertas e notificar como verdadeiro. Tempo: 42 segundos. Cliques: 6. Observações: A interface é direta, mas a distinção das cores nos botões de verdadeiro/falso poderia ser mais evidente para evitar hesitações durante uma emergência.
- **André Noronha:** Tarefa: Alterar password do utilizador nas definições. Tempo: 50 segundos. Cliques: 8. Observações: Boa fluidez. No entanto, sentimos falta de um ícone de visibilidade na password ("olho") para confirmar o que está a ser digitado (funcionalidade que implementámos nativamente na GRAB!T).
- **Filipe Lino:** Tarefa: Criar nova câmara e verificar o seu estado. Tempo: 1 minuto e 10 segundos. Cliques: 12. Observações: O fluxo é bom, mas após o registo o utilizador não é reencaminhado diretamente para o ecrã principal, forçando a realização de um novo Login manual. No geral, um excelente projeto ao nível de base de dados.

Feedback recebido da equipa LynxSegur sobre a GRAB!T: Os colegas avaliaram a nossa aplicação com tempos médios muito positivos (entre 1m00s e 1m33s) para concluir fluxos completos de encomenda. Destacaram a qualidade da interface gráfica em Jetpack Compose, deixando sugestões focadas na expansão de rodapés nalguns ecrãs. Este feedback servirá de base para futuras atualizações.

11 Página web da aplicação e screencast

Foi programada uma página web Landing Page promocional desenhada em conformidade visual com o tema principal da aplicação mobile (fundo escuro e fontes/botões em tom esverdeado). O website divulga eficientemente os objetivos do projeto, expõe a secção "Como Funciona?" (Escolhe, Pede, Desfruta), detalha os nossos compromissos (Segurança, Opções Rápidas) e apresenta a formação da equipa técnica (Gustavo, Filipe e André).

Adicionalmente, integrámos na entrega o screencast interativo: um vídeo de demonstração com a duração aproximada de 3 minutos. Gravado num emulador Android, exhibe em ambiente real a rapidez nativa, os ecrãs de registo, os catálogos dinâmicos e a comunicação Backend-Frontend idealizada na GRAB!T.

Ficheiros: [GitHub](#)

Link Screencast: [Drive](#)

12 Conclusão

O planeamento e construção da aplicação GRAB!T representou uma jornada técnica muito gratificante. A transição da solução conceptual inicial de ambiente desktop para a plataforma Android validou as opções do grupo face às necessidades de mobilidade que o serviço de delivery exige.

O grande sucesso da implementação assentou no cruzamento de valências entre a equipa: do rigor na documentação e escalabilidade em SQL (Room), passando pela criação gráfica intuitiva, culminando na avançada programação através da linguagem Kotlin e da framework declarativa Jetpack Compose. Esta combinação garantiu uma aplicação veloz, segura (com sólidas lógicas de prevenção de erros) e altamente interativa (gestão de estados reativos).

As avaliações efetuadas a utilizadores comprovam a fiabilidade da estrutura elaborada na unidade curricular. Numa perspetiva de continuidade comercial, a base do projeto encontra-se inteiramente estruturada e recetiva à futura implementação de APIs geográficas nativas (Google Maps) e integrações de portais reais de pagamento eletrónico.